

ChatGPT



Sorting - Part 1 | Selection Sort, Bubble Sort, Insertion Sort | Strivers A2Z DSA Course

```
code > G coderun.cpp > main()
9      }
10     }
11     int temp = arr[mini];
12     arr[mini] = arr[i];
13     arr[i] = temp;
14 }
15 }
16 void bubble_sort(int arr[], int n) {
17     for(int i = n-1; i >= 0; i--) {
18         for(int j = 0; j <= i-1; j++) {
19             if(arr[j] > arr[j+1]) {
20                 int temp = arr[j+1];
21                 arr[j+1] = arr[j];
22                 arr[j] = temp;
23             }
24         }
25     }
26 }
27 int main() {
28     int n;
29     cin >> n;
30     int arr[n];
31     for(int i = 0; i < n; i++) cin >> arr[i];
32     bubble_sort(arr, n);
33     for(int i = 0; i < n; i++) {
34         cout << arr[i] << " ";
35     }
36     return 0;
}
```

input.txt

```
1 6
2 13 46 24 52 20 9
3
```

output.txt

```
1 6 13 20 24 46 52
```

```
code > G coderun.cpp > bubble_sort(int [], int)
8      if(arr[j] < arr[mini]) {
9          mini = j;
10     }
11     int temp = arr[mini];
12     arr[mini] = arr[i];
13     arr[i] = temp;
14 }
15 }
16 void bubble_sort(int arr[], int n) {
17     for(int i = n-1; i >= 0; i--) {
18         int didSwap = 0;
19         for(int j = 0; j <= i-1; j++) {
20             if(arr[j] > arr[j+1]) {
21                 int temp = arr[j+1];
22                 arr[j+1] = arr[j];
23                 arr[j] = temp;
24                 didSwap = 1;
25             }
26         }
27         if(didSwap == 0) {
28             break;
29         }
30         cout << "runs\n";
31     }
32 }
33 int main() {
34     int n;
```

input.txt

```
1 6
2 6 5 4 3 2 1
3
```

output.txt

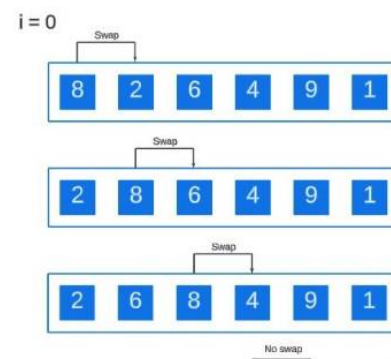
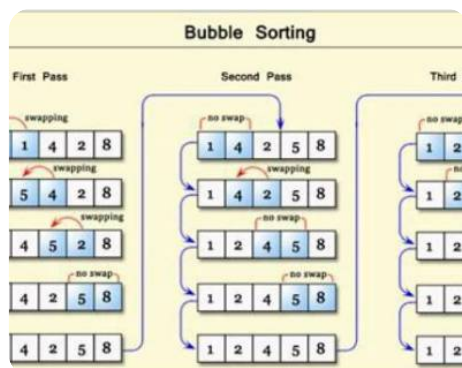
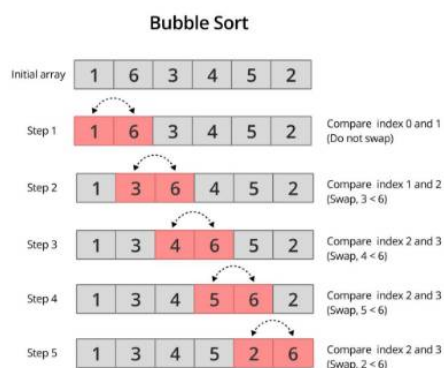
```
1 runs
2 runs
3 runs
4 runs
5 runs
6 1 2 3 4 5 6
```

Teach me Bubble sort and its java implementation

Compare with other AI ⚡

Absolutely. Let's learn **Bubble Sort** from intuition → dry run → Java code → optimization → complexity.

Your screenshots show exactly the standard Bubble Sort idea, including the **didSwap** optimization.



1. What is Bubble Sort?

Suppose we have:

[5, 4, 3, 2, 1]

Bubble Sort repeatedly compares **two adjacent elements**:

```
5 4 3 2 1
↑  ↑
```

If they are in the wrong order, swap them.

```
5 > 4 → swap
4 5 3 2 1
```

Then move one position forward:

```
4 5 3 2 1
↑  ↑
```

5 > 3, so swap:

```
4 3 5 2 1
```

Keep doing this.

The important observation is:

After one complete pass, the largest remaining element "bubbles" to the end.

That's why it's called **Bubble Sort**.

2. Let's dry-run it

Take:

```
[5, 4, 3, 2, 1]
```

Pass 1

Compare adjacent elements:

```
5 4 → swap → 4 5 3 2 1
5 3 → swap → 4 3 5 2 1
5 2 → swap → 4 3 2 5 1
5 1 → swap → 4 3 2 1 5
```

Now:

```
[4, 3, 2, 1, 5]
```

Notice that **5 is in its final position**.

Pass 2

We don't need to touch 5 anymore.

```
4 3 → 3 4 2 1 5
4 2 → 3 2 4 1 5
4 1 → 3 2 1 4 5
```

Now:

```
[3, 2, 1, 4, 5]
```

Pass 3

```
3 2 → 2 3 1 4 5
3 1 → 2 1 3 4 5
```

Pass 4

```
2 1 → 1 2 3 4 5
```

Sorted:

```
[1, 2, 3, 4, 5]
```

3. The core idea in one line

This is the most important line:

```
Java
if (arr[j] > arr[j + 1])
```

Meaning:

"Are these two adjacent elements in the wrong order?"

If yes, swap them.

Java

```
int temp = arr[j];
arr[j] = arr[j + 1];
arr[j + 1] = temp;
```

For example:

```
arr[j]    arr[j+1]
  5        3
  ↓        ↓
5 > 3 → swap
3         5
```

4. Java implementation

Here's the basic version:

Java

```
public class Main {
    static void bubbleSort(int[] arr) {
        int n = arr.length;
        for (int i = n - 1; i >= 0; i--) {
            for (int j = 0; j < i; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }

    public static void main(String[] args) {
        int[] arr = {5, 4, 3, 2, 1};
        bubbleSort(arr);
        for (int x : arr) {
            System.out.print(x + " ");
        }
    }
}
```

Output:

```
1 2 3 4 5
```

5. Understand the two loops

This is where Bubble Sort usually becomes confusing.

Java

```
for (int i = n - 1; i >= 0; i--) {
    for (int j = 0; j < i; j++) {
        if (arr[j] > arr[j + 1]) {
            // swap
        }
    }
}
```

Think of **i** as telling us:

"Up to where do I need to compare?"

For an array of size 5:

```
[5 4 3 2 1]
```

Initially:

```
i = 4
```

So:

Java

```
j < 4
```

We compare:

```
j=0 → arr[0] with arr[1]
j=1 → arr[1] with arr[2]
j=2 → arr[2] with arr[3]
j=3 → arr[3] with arr[4]
```

After that pass, the largest element is at index 4.

So next:

```
i = 3
```

We only need:

```
j = 0, 1, 2
```

because index 4 is already sorted.

Therefore:

```
Pass 1 → last 1 element sorted
Pass 2 → last 2 elements sorted
Pass 3 → last 3 elements sorted
...
```

That's why the inner loop gets shorter.

6. The didSwap optimization from your screenshot

Your second screenshot contains:

```
Java
int didSwap = 0;
```

and later:

```
Java
if (arr[j] > arr[j + 1]) {
    int temp = arr[j];
    arr[j] = arr[j + 1];
    arr[j + 1] = temp;

    didSwap = 1;
}
```

Then:

```
Java
if (didSwap == 0) {
    break;
}
```

This is a **very useful optimization**.

Why?

Imagine the array is already sorted:

```
[1, 2, 3, 4, 5]
```

First pass:

```
1 2 → no swap
2 3 → no swap
3 4 → no swap
4 5 → no swap
```

No swap happened.

Therefore:

```
Java
didSwap == 0
```

So we know:

The array was already sorted. Stop!

Without this optimization, Bubble Sort would continue making unnecessary passes.

7. Optimized Java version

This is the version I recommend remembering for interviews:

```
Java
public class Main {

    static void bubbleSort(int[] arr) {
        int n = arr.length;
        for (int i = n - 1; i >= 1; i--) {
            boolean didSwap = false;
            for (int j = 0; j < i; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    didSwap = true;
                }
            }
            if (!didSwap) {
                break;
            }
        }
    }

    public static void main(String[] args) {
        int[] arr = {5, 4, 3, 2, 1};
        bubbleSort(arr);
        for (int x : arr) {
            System.out.print(x + " ");
        }
    }
}
```

Output:

```
1 2 3 4 5
```

8. Why boolean instead of int?

Your screenshot uses:

```
Java
int didSwap = 0;
```

and:

```
Java
didSwap = 1;
```

That's perfectly understandable, but Java has a better type for this:

Java

```
boolean didSwap = false;
```

Then:

Java

```
didSwap = true;
```

And:

Java

```
if (!didSwap) {
    break;
}
```

This reads almost like English:

If we **didn't swap**, break.

9. Complexity

For an array of n elements:

Worst case

Example:

```
[5, 4, 3, 2, 1]
```

Lots of swapping is required.

Time:

```
 $O(n^2)$ 
```

Average case

Also:

```
 $O(n^2)$ 
```

Best case

With the **didSwap** optimization:

```
[1, 2, 3, 4, 5]
```

Only one pass is needed.

Time:

```
0(n)
```

Space

Bubble Sort sorts the array **in-place**, so:

```
Space = 0(1)
```

10. The interview explanation

If an interviewer asks:

"Explain Bubble Sort."

A strong answer is:

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. After every pass, the largest unsorted element reaches its correct position at the end of the array. The inner loop therefore becomes smaller after each pass. Its average and worst-case time complexity is $O(n^2)$, while with the swapped flag its best case is $O(n)$. It uses $O(1)$ extra space.

That's enough for a typical DSA interview.

11. The mental model you should remember

Don't try to memorize the code first. Remember these **3 rules**:

Rule 1 — Compare neighbors

```
Java
arr[j] and arr[j + 1]
```

Rule 2 — Swap if left > right

```
Java
if (arr[j] > arr[j + 1])
```

Rule 3 — Each pass puts the largest element at the end

```
[5 4 3 2 1]
      ↑
    largest
```

Then:

```
[4 3 2 1 | 5]
      sorted
```

Then:

```
[3 2 1 | 4 5]  
      sorted
```

Then:

```
[2 1 | 3 4 5]
```

Then:

```
[1 | 2 3 4 5]
```

That's Bubble Sort. Once that picture is clear, the nested loops become much easier to understand.
ChatGPT can make mistakes. Check important info.